

Chapter 4

Scheduling Models

4.1 Introduction

Matching supply and demand is commonly referred to as an atemporal event, as if markets clear instantaneously. Even in the most sophisticated financial markets, matching supply and demand is far from an instantaneous affair where order matters and makes a significant difference in the outcome. For example, the price difference between transactions can amount to very large amounts of money in a matter of milliseconds.

In physical markets, the problem is even more pronounced. For example, in the warehouse setting considered in the previous section, demand is fulfilled in a specific order that if not properly optimized can lead to congestion, lost sales and waste of resources. Many of these problems are very simple to describe but difficult to solve: the order of packaging boxes in a warehouse, the order of manufacturing different products, or deciding the schedules of workers to fulfill a productive activity.

Similar to other chapters when we studied the demand, scheduling can be studied as a time-space matching problem. We start by studying what would be the *space* variant (which could also be seen as discrete time). Then, we will move to consider the problem when time is a random variable and finally consider both problems simultaneously in future chapters.

4.2 A set covering problem

A lot of scheduling problems take the form of an abstract problem: to *cover* a set at minimum cost. Here, set can be understood in a general way as a demand that extends over either time or space, and certain resources are available to satisfy it. For example, a fixed number of trucks need to be assigned to cover deliveries in a city, or workers need to be assigned tasks to complete a production process. Many of these problems are combinatorial in nature, meaning that finding

optimal solutions is in general hard, except in the simplest cases.

Like in previous chapters, we start with the simplest formulation that we will extend to other special cases. Consider a demand vector $\mathbf{D}$, that like in previous chapters can represent the demand for multiple locations or time-periods, the demand commonly has a task interpretation, let the dimension of the demand vector be N . Let $\mathcal{S}$ be a set of different J schedules available to fulfill some task. Moreover, define the matrix A with dimensions $N \times J$, and elements $a_{ij} = 1$ if task i is covered by schedule j . Let x be a decision variable (a vector of size J) in some space of feasible assignments $\mathcal{X}$ (for example, $\mathcal{X} = \{0, 1\}^J$). The scheduling problem normally takes the form:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & Ax^\top \geq b\mathbf{E}(\mathbf{D}), \\ & x \in \mathcal{X}. \end{aligned} \tag{4.1}$$

See Figure 4.1 to see the structure of the problem.

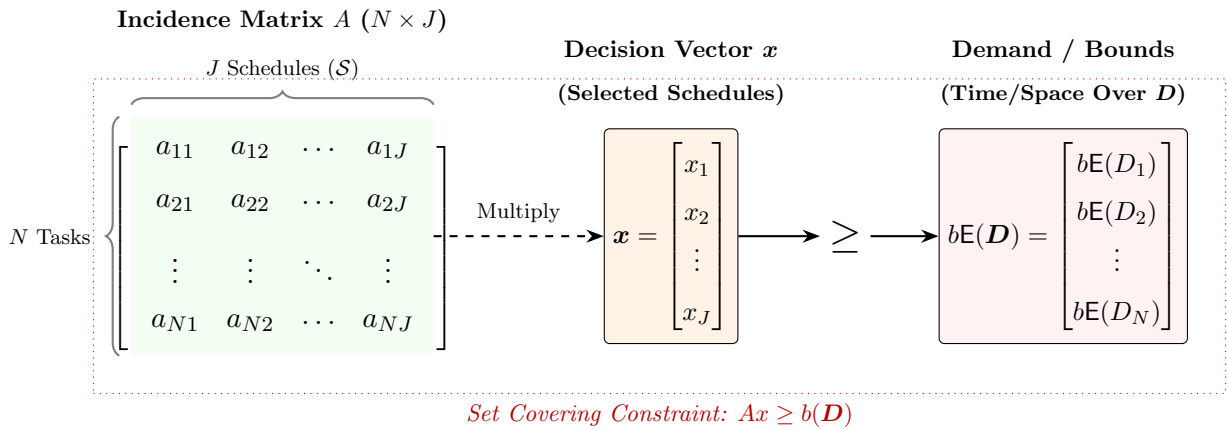


Figure 4.1: Structural breakdown of the matrix-vector alignment in the set covering scheduling formulation, where available schedules ($\mathcal{S}$) map onto structural demands over a space-time domain.

We illustrate next with a couple of examples the flexibility of this formulation.

Example 4.1 (Shift Scheduling). A warehouse wants to schedule workers to process packages in a 24 hour basis. Every hour, the warehouse has a demand of packages represented by a vector $\mathbf{D} = (D_0, \dots, D_{23})$. The warehouse has a headcount of H workers that it has to allocate for 8-hour consecutive shifts. That is, there are possible 24 different shifts each starting every hour. A worker can process b packages per hour, therefore the need of workers is in expected value $b\mathbf{E}(\mathbf{D})$. Let A be a matrix with 24 columns and 24 rows and entries $a_{tj} = 1$ if schedule j covers hour t . Moreover, let x be a vector with $J = 24$ elements denoting the headcount of workers

assigned to each schedule. The manager wants to maximize the expected buffer between supply and demand across all hours of the day.

The first step to solve the problem is to define the objective function $f(x)$. In this case, the gap between supply and demand at every hour is $Ax^\top - bE(\mathbf{D})$, as $Ax^\top$ is the number of scheduled workers every hour. Then, the minimum buffer between supply and demand is $v = \min_t \{(Ax^\top)_t - bE(D_t)\}$. As we want to make this as large as possible, $f(x) = -v$ achieves the goal of increasing the gap between supply and demand as much as possible between supply and demand. Finally, the constraint $|x| = H$ achieves the goal of assigning all workers (recall that for vectors $|x| = \sum_{j=1}^J |x_j|$). The final formulation results in the following MIP:

$$\begin{aligned} \max \quad & v \\ \text{s.t.} \quad & Ax^\top - bE(\mathbf{D}) \geq v, \\ & v \in \mathbb{R}, x \in \mathbb{N}^J, |x| = H. \end{aligned} \tag{4.2}$$

As a follow-up, the business managers want to know instead the minimum amount of workers such that they can cover their operation in expected value.

This can be achieved by modifying the function $f(x)$ to $|x|$, that we previously shown is the total number of workers. Also, the constraint that represents that the demand is covered at all hours in expected value is $Ax^\top \geq bE(\mathbf{D})$. Leading to the alternative formulation:

$$\begin{aligned} \min \quad & |x| \\ \text{s.t.} \quad & Ax^\top \geq bE(\mathbf{D}), \\ & x \in \mathbb{N}^J. \end{aligned} \tag{4.3}$$

The last example can also be easily modified when each schedule has a cost vector c associated to each entry of x , and minimizing $f(x) = cx^\top$. When the cost of all schedules is the same, or the cost is proportional to number of hours (at the same rate), then the optimal value x^* is the same as the previous problem with $f(x) = |x|$.

4.2.1 Data Uncertainty

We revisit a technique in the warehouse allocation problem to deal with uncertainty in the data matrix A . In the previous formulation. The matrix A had entries $a_{ij} = 1$ whenever schedule j is covering the period or location i . The 1 has a direct meaning that every resource allocated on that schedule is going to *certainly* be allocated. In real life this is rarely the case as there are many occasions when an allocated resource might not fully work: in the case of people it

can be because of a curve of productivity (that is not uniform across all hours of the shift). In the case of a machine it can be because of a failure that occurs with a small probability. Here we provide a data-driven framework to deal with these kind of uncertainty:

Consider each row of the matrix A as a multi-variate normal random variable a_i with mean $\bar{a}_i$ and covariance matrix Σ_i . For example, in the productivity curve case where each hour of the shift has their own productivity level $a_i \in [0, 1]^J$ is a random vector with the different productivity levels. The mean and the variance could be estimated from a sample size of n observations for example. Each constrain for the demand satisfaction can be transformed into the probabilistic constraint:

$$\mathbf{P}(a_i x^\top \geq b D_i) \geq 1 - \eta, \quad (4.4)$$

Assuming the demand D_i is another independent random variable, the linear combination $a_i x^\top - b D_i$ is another random variable. The mean of this random variable is $\bar{a}_i x^\top - b \mathbf{E}(D_i)$ and its variance is $x \Sigma_i x^\top + b^2 \text{Var}(D_i)$. Using the usual trick with the inverse of the normal random variable trick seen in other chapters, we have that $\mathbf{P}(a_i x^\top \geq b D_i)$ is equal to $\mathbf{P}(Z \geq -(\bar{a}_i x^\top - b \mathbf{E}(D_i)) / \sqrt{x \Sigma_i x^\top + b^2 \text{Var}(D_i)})$ or $1 - \mathbf{P}(Z \leq -(\bar{a}_i x^\top - b \mathbf{E}(D_i)) / \sqrt{x \Sigma_i x^\top + b^2 \text{Var}(D_i)})$. Making the inequality equal to $\mathbf{P}(Z \leq -(\bar{a}_i x^\top - b \mathbf{E}(D_i)) / \sqrt{x \Sigma_i x^\top + b^2 \text{Var}(D_i)}) \leq \eta$. Taking the inverse cdf of the normal yields the equivalent constraint:

$$\bar{a}_i x^\top + \Phi^{-1}(\eta) \sqrt{x \Sigma_i x^\top + b^2 \text{Var}(D_i)} \geq b \mathbf{E}(D_i), \quad (4.5)$$

Thematically, this constraint follows a similar pattern through the book that hedging the risk of allocating resources not only involves demand risk, but also the risk associated to allocating the resources.

Next, we need to transform this constraint into a tractable formulation as the term $x \Sigma_i x^\top$ is not linear in x , thus, not directly a MIP. A powerful paradigm for these kind of problems is known as *Second-order Cone Programming* or SOCP. Note that $\sqrt{x \Sigma_i x^\top + b^2 \text{Var}(D_i)}$ is equivalent to $\|(\Sigma_i^{1/2} x^\top, \sqrt{\text{Var}(b D_i)})\|_2$, where $\Sigma_i^{1/2}$ is a matrix such that $\Sigma_i^{1/2} (\Sigma_i^{1/2})^\top = \Sigma_i$, for example, think of Cholesky decomposition. Using this fact, the original scheduling program can be rewritten as:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & -\Phi^{-1}(\eta) \left\| \begin{pmatrix} \Sigma_i^{1/2} x^\top \\ \sqrt{\text{Var}(b D_i)} \end{pmatrix} \right\|_2 \leq \bar{a}_i x^\top - b \mathbf{E}(D_i) \text{ for } i = 1, \dots, N, \\ & x \in \mathcal{X}. \end{aligned} \quad (4.6)$$

When $f(x)$ is an affine function, this optimization program can be solved by a MISOCP solver. This formulation is flexible to accomodate an interesting range of applications: for example, a finite sample observation of $\bar{a}_i$

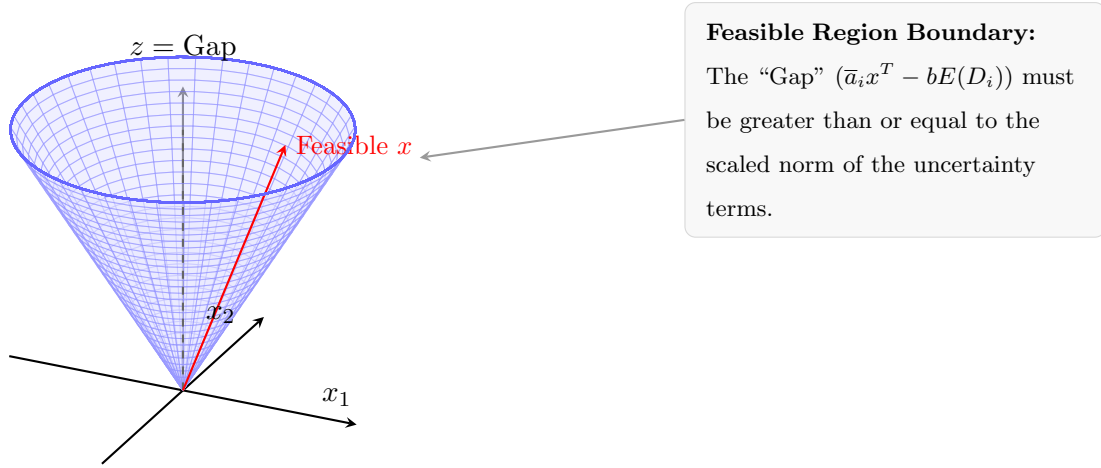


Figure 4.2: The Lorentz Cone representation. The vertical axis represents the supply-demand gap, while the interior represents the feasible set for the SOCP constraint.

4.3 Production Scheduling

We change pace by describing a different kind of scheduling models closely related to closest route problems, and that provide a useful : Suppose a production planning problem over a horizon of T periods in the future. We start with the simplest case when the demand is known, that is, at every period the demands $d_1, d_2, \dots, d_T$ are known in advance. Define $c(s, t)$ as the production and holding costs of periods $s, \dots, t$ for $s \leq t \in \{0, \dots, T\}$ (inclusive of all periods in the range and the starting and endpoint) so that demand is satisfied exactly.

For example, $c(0, 4)$ includes the setup cost for production, plus the variable cost of producing for demands $d_1, \dots, d_4$ and the cost of storing the demand all the way to period 4 (starting production at time 0). The key observation is that once this production is finished, starting at period 5, the system *renews* and the past production decision can be ignored. As an example, this cost can be a setup cost K_0 of starting production at period 0, plus a variable cost $\sum_{t=1}^4 v_0(d_t)$ and holding costs $h_0(d_1, \dots, d_4)$ until the respective demand period. Then, $c(0, 4)$ could be:

$$c(0, 4) = K_0 + \sum_{t=1}^4 v_0(d_t) + h_0(d_1, \dots, d_4), \quad (4.7)$$

Nonetheless, this is not the only possible specification for the costs. The important part is that all these costs are deterministic and can be computed.

This structure of the problem allows us to use a powerful paradigm to solve the problem. Let $c(t)$ be defined as the minimum cost of meeting demand for periods $t, \dots, T$. Then, we have the following recursion:

$$c(s) = \min_{t \geq s} [c(s, t) + c(t + 1)], \quad (4.8)$$

This recursion implies a similar structure to a shortest-path-problem in a graph with nodes

$V = \{0, 1, \dots, T\}$ and edges $E = \{(s, t) : s \leq t \in \{0, \dots, T\}\}$ with distances $c(s, t)$. The technique to solve the problem is to start with $c(T + 1) = 0$, and solve recursively backwards until arriving to $c(0)$. The solution is a shortest-path between nodes 0 and T of the graph. Each node visited represents the periods where production should take place. When the demands are random and the costs have non-linearities. The model reduces to the dynamic inventory management case studied in Chapter 2.

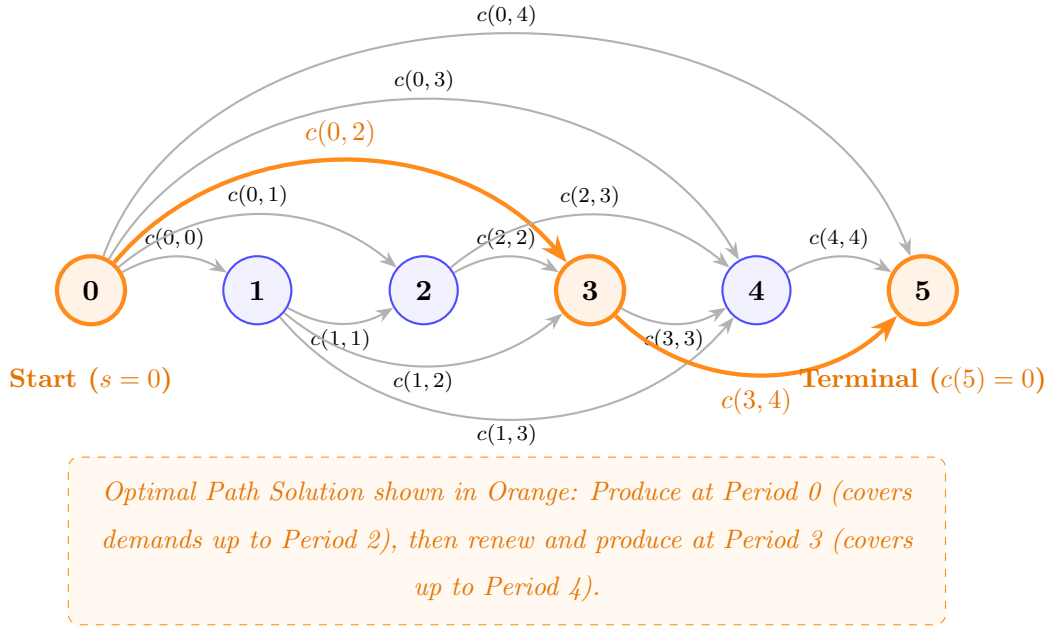


Figure 4.3: Production scheduling mapped as a shortest path network over a horizon where $T = 4$. Edge weights $c(s, t)$ represent setup, variable production, and holding costs. Visited nodes on the optimal shortest path (highlighted in orange) signal the optimal periods where regeneration takes place and production runs must occur.

4.4 A knapsack problem

Lastly, we cover a problem that while not necessarily deals with scheduling in a nominal way, it is closely related as it deals with fitting objects in a limited space and can be interpreted in many ways: for example choosing jobs to perform in a limited time, or choosing investments subject a budget. This problem is known as the *Knapsack Problem*. The most common variations of this problem can be formulated in the following setting. Imagine a knapsack (or a warehouse) where a series of objects $i = 1, \dots, N$ has a profit or benefit expressed as $f_i(x_i)$ and a capacity of the knapsack consumed $c_i(x_i) \geq 0$. This capacity is assumed to be positive and increasing in x_i . $x_i \in \mathcal{X}$ either expresses a quantity or indicator of how much of each object is used in the knapsack. The objective is to pack the knapsack with the objects that have maximum benefit,

subject to not exceeding the total capacity of the knapsack.

$$\begin{aligned} \max \quad & \sum_{i=1}^N f_i(x_i) \\ \text{s.t.} \quad & \sum_{i=1}^N c_i(x_i) \leq C, \\ & x_i \in \mathcal{X} \text{ for } i = 1, \dots, N. \end{aligned} \tag{4.9}$$

This is again a difficult combinatorial problem, as the greedy heuristics of sorting objects with respect to the metric $f_i(x_i)/c_i(x_i)$ might not lead to an optimal assortment of objects.

This problem can be solved in multiple ways: For example, the problem can be expressed as a Mixed-Integer Program and be solved directly with solvers. Although, when the benefit function $f_i(x_i)$ is non-linear this is not the case, see Figure 4.4 for an example.

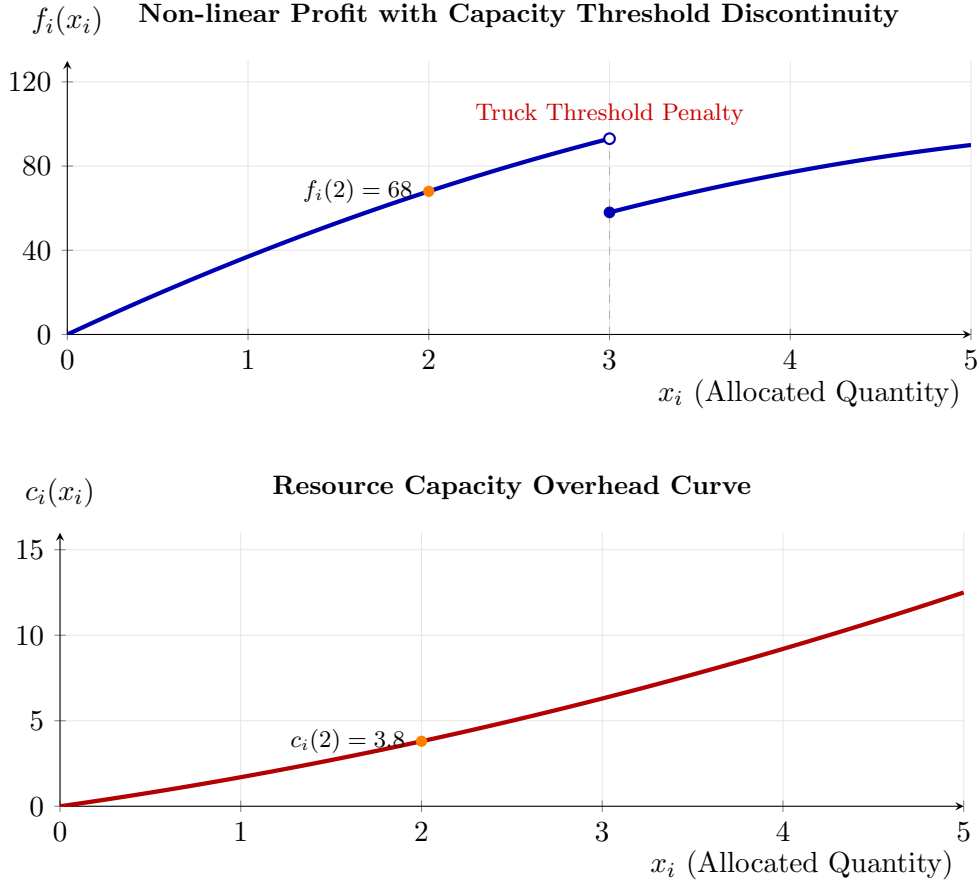


Figure 4.4: Visualizing a non-linear knapsack problem where the profit function incorporates a structural step-down penalty. When the allocated allocation quantity crosses the threshold ($x_i = 3$), the sudden acquisition cost of an additional delivery vehicle drops total net profit before marginal returns resume growth.

For a general benefit function, we present an alternative procedure to achieve the same

result. This procedure is called *forward-recursion* or *Dynamic Programming* depending on the textbook. In fact, this is similar to the production example or the dynamic inventory model presented in Chapter 2). First define the optimal benefit for a knapsack of capacity $k \leq C$ using the first $j \leq N$ objects:

$$F_j(k) := \max_{\sum_{i=1}^j c_i(x_i) \leq k, x_i \in \mathcal{X}} \left(\sum_{i=1}^j f_i(x_i) \right), \quad (4.10)$$

The solution of the problem is by definition $F_N(C)$. Now, the trick is again to create a one-step recursion that relates these quantities to their predecessors (as we are going forwards instead of backwards). This recursion is:

$$F_j(k) = \max_{x_j \in \mathcal{X}} \{f_j(x_j) + F_{j-1}(k - c_j(x_j))\}, \quad (4.11)$$

The intuition is clear: the maximum benefit of a knapsack of capacity k using the first j objects is equal to the largest benefit of the object j , that is, $f_j(x_j)$ plus the optimal benefit of a knapsack using the previous $j - 1$ objects with capacity $k - c_j(x_j)$. The only thing left is to define the boundary conditions to start the recursion. These can be defined as:

$$F_0(k) = \begin{cases} 0, & \text{for } k \geq 0 \\ -\infty, & \text{for } k < 0. \end{cases} \quad (4.12)$$

Then, the solution can be computed recursively by filling out the elements $F_j(k)$ in a forward fashion. The underlying assumption is that the capacity function c_i is a non-decreasing function of x_i . And the number of operations is of order $O(CN|\mathcal{X}|)$. See Figure 4.5 for a schematic of the DP recursion.

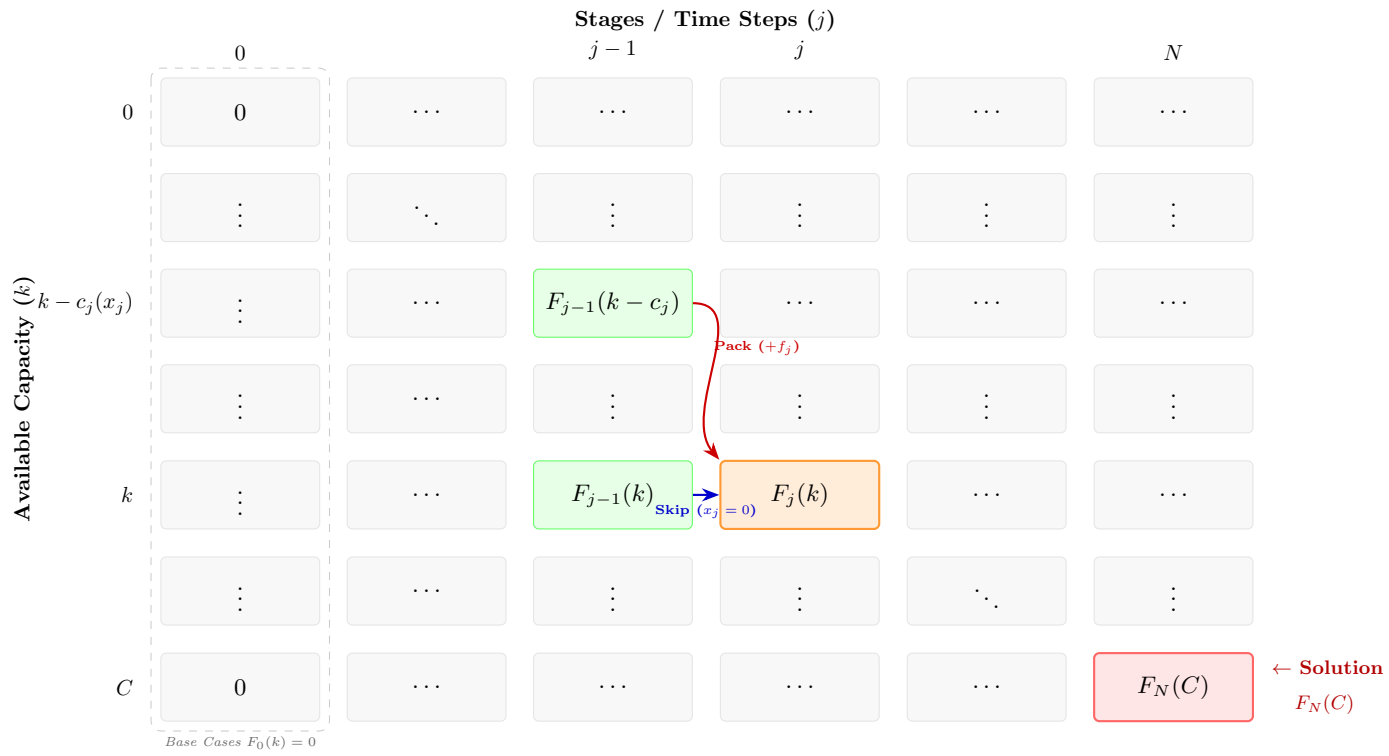


Figure 4.5: Matrix representation of the forward-recursion Dynamic Programming state space for the Knapsack problem. Columns represent sequential decision horizons (stages/time), while rows represent the remaining sub-problem capacity. Computing the cell $F_j(k)$ evaluates the maximum option between state transitions in stage $j - 1$.

